

Emacs and Vim

Emacs and *Vim* are two popular editors favored by many experienced programmers, because they're designed so you can use them without your hands ever having to leave the keyboard. This makes writing, reading, and modifying code very efficient, once you learn how the editor works. It also means both editors have a fairly steep learning curve. *Vim* is included on most Linux and macOS machines, and both *Emacs* and *Vim* can be run entirely inside a terminal. For this reason, they're often used to write code on servers through remote terminal sessions.

Programmers will often recommend that you give them a try, but many proficient programmers forget how much new programmers are already trying to learn. It's good to be aware of these editors, but you should hold off on using them until you're comfortable working with code in a more user-friendly editor that lets you focus on learning to program, rather than learning to use an editor.

PyCharm

PyCharm is a popular IDE among Python programmers because it was built to work specifically with Python. The full version requires a paid subscription, but a free version called the *PyCharm Community Edition* is also available, and many developers find it useful.

If you try *PyCharm*, be aware that, by default, it sets up an isolated environment for each of your projects. This is usually a good thing, but it can lead to unexpected behavior if you don't understand what it's doing for you.

Jupyter Notebooks

Jupyter Notebook is a different kind of tool than traditional text editors or IDEs, in that it's a web app primarily built of blocks; each block is either a code block or a text block. The text blocks are rendered in Markdown, so you can include simple formatting in your text blocks.

Jupyter Notebooks were developed to support the use of Python in scientific applications, but they have since expanded to become useful in a wide variety of situations. Rather than just writing comments inside a *.py* file, you can write clear text with simple formatting, such as headers, bulleted lists, and hyperlinks in between sections of code. Every code block can be run independently, allowing you to test small pieces of your program, or you can run all the code blocks at once. Each code block has its own output area, and you can toggle the output areas on or off as needed.

Jupyter Notebooks can be confusing at times because of the interactions between different cells. If you define a function in one cell, that function is available to other cells as well. This is beneficial most of the time, but it can be confusing in longer notebooks and if you don't fully understand how the Notebook environment works.

If you're doing any scientific or data-focused work in Python, you'll almost certainly see *Jupyter Notebooks* at some point.